UNIVERSIDAD DE MANIZALES
FACULTAD DE CIENCIAS E INGENIERÍAS
INGENIERÍA EN ANALÍTICA DE DATOS

DOCUMENTO DE DECISIONES DE DISEÑO
Actividad Colaborativa N°3: Proyecto Final
Gestión de Tasas Aeroportuarias Aeropuertos

Autores:
Juan David Perdomo Rojas
Legna Liliana Ferreira Rosas

Asignatura:
Programación I

Manizales, Colombia
noviembre de 2026

VIGILADO Por el Ministerio de Educación Nacional
Campusvirtual Universidad de Manizales
ISO 9001:2008 MSC-57117129 www.fbcert.services www.certidata.info
Acreditación Institucional de Alta Calidad Resolución 4792 del 15 de mayo de 2019
UNIVERSIDAD DE MANIZALES

1. Introducción

Este documento presenta y justifica las decisiones de diseño y de desarrollo del proyecto final de Programación I: un sistema de información en Python llamado *Gestión de Tasas Aeroportuarias Aeropuertos*. Aplica de manera integrada, los conceptos de programación orientada a objetos estudiados en el módulo: encapsulamiento, herencia, clase abstracta, sobrescritura, polimorfismo, composición y manejo de excepciones personalizadas.

El problema que aborda es de carácter general: muchas organizaciones cobran un servicio a partir de lo que otros declaran, y esa declaración puede estar incompleta, equivocada o simplemente no existir. En el contexto de un aeropuerto, las aerolíneas declaran los pasajeros de cada vuelo, mientras que el aeropuerto conoce los vuelos que realmente operaron. El sistema compara ambos registros, detecta cada diferencia, la clasifica en uno de tres tipos, estima el valor que está en riesgo y entrega un informe priorizado con la acción recomendada para cada caso. El propósito no es reproducir ningún procedimiento real, sino usar este escenario para demostrar un diseño orientado a objetos coherente y verificable; por eso todos los datos, nombres, tarifas y plazos del proyecto son ficticios.

El documento sigue el orden del proceso de construcción: describe la solución, desarrolla las decisiones de diseño relacionándolas con el código entregado, declara el uso de herramientas de inteligencia artificial y cierra con las conclusiones y las fuentes consultadas.

2. Descripción General de la Solución Implementada

El sistema se organiza en catorce archivos de Python, tres archivos CSV de datos ficticios y un conjunto de pruebas automáticas. Cada archivo tiene una responsabilidad reconocible: hallazgo.py define la clase abstracta y cada clase hija ocupa su propio archivo; aeropuerto.py, vuelo.py, aerolinea.py y declaracion.py modelan los datos; informe_conciliacion.py, conciliador.py y cargador_csv.py implementan el proceso; excepciones.py reúne las excepciones propias, validaciones.py las funciones de validación y main.py es el programa principal.

El flujo de trabajo tiene cuatro pasos. Primero, CargadorCSV lee vuelos_operados.csv y declaraciones.csv y construye los objetos Aerolinea, Vuelo y Declaracion. Segundo, Conciliador cruza las declaraciones contra los vuelos operados usando la clave (aerolínea, código de vuelo, fecha) y le pide a InformeConciliacion que registre cada diferencia encontrada. Tercero, el informe crea los hallazgos —objetos de tres clases que heredan de Hallazgo—, los ordena por valor en riesgo y totaliza el resultado según su impacto. Cuarto, main.py demuestra el polimorfismo y los casos excepcionales.

Con los datos de ejemplo (un aeropuerto ficticio con tarifa de $20.000 por pasajero, tres aerolíneas, doce vuelos operados y once declaraciones) el sistema detecta seis diferencias: dos vuelos sin declaración, una declaración sin vuelo y tres diferencias de pasajeros. El valor en riesgo asciende a $3.440.000 por faltante de recaudo y a $900.000 por posible cobro en exceso; ambos se informan por separado porque significan cosas

VIGILADO Por el Ministerio de Educación Nacional
Campusvirtual Universidad de Manizales
ISO 9001:2008 MSC-57117129 CERTIFIED www.fbcert.services www.certidata.info
Acreditación Institucional de Alta Calidad Resolución 4192 del 15 de mayo de 2019
UNIVERSIDAD DE MANIZALES

opuestas. La figura 1, incluida como anexo al final de este documento, muestra el diagrama UML, generado a partir del propio código para que refleje lo implementado.

3. Desarrollo y Justificación de las Decisiones de Diseño

3.1. El problema que resuelve el sistema y su delimitación

El sistema resuelve un problema específico: verificar lo que declaran terceros contra los registros propios de quien cobra. Cada declaración que presenta una aerolínea indica cuántos pasajeros viajaron en un vuelo; cada vuelo operado indica cuántos pasajeros transportó realmente. Cuando ambos datos no coinciden hay una diferencia, y esa diferencia puede ser de tres clases: el vuelo ocurrió pero nadie lo declaró, alguien declaró un vuelo que no consta como operado, o ambos existen pero los pasajeros no coinciden. Para cada una el sistema calcula cuánto dinero está en juego y qué debería hacer la persona responsable.

Se delimitó el alcance de forma completa: el sistema trabaja con un solo aeropuerto, no calcula tarifas reales, no consulta bases de datos y no pretende representar un proceso oficial. Esta delimitación ha permitido dedicar el esfuerzo a lo que la actividad evalúa —el diseño orientado a objetos.

3.2. Clases principales y responsabilidad de cada una

Se definieron once clases del dominio, una de ellas abstracta, más una jerarquía de excepciones. La regla que seguimos fue que cada clase pudiera describirse con una sola frase; cuando una clase necesitaba dos frases con "y", separamos sus responsabilidades. La tabla 1 resume el resultado.

Tabla 1. Clases del sistema y su responsabilidad

Clase	Archivo	Responsabilidad
Hallazgo (abstracta)	hallazgo.py	Representa una diferencia detectada y define cómo se valora, qué impacto tiene y qué acción se recomienda.
VueloSinDeclaracion	vuelo_sin_declaracion.py	Hallazgo de un vuelo operado que ninguna aerolínea declaró.
DeclaracionSinVuelo	declaracion_sin_vuelo.py	Hallazgo de una declaración cuyo vuelo no figura como operado.
DiferenciaPasajeros	diferencia_pasajeros.py	Hallazgo de un vuelo que existe en ambos lados con pasajeros distintos.
Aeropuerto	aeropuerto.py	Identifica el aeropuerto, guarda su tarifa ficticia por pasajero y estima cobros.
Vuelo	vuelo.py	Vuelo realmente operado: es el dato propio de quien cobra.
Aerolinea	aerolinea.py	Identifica a la aerolínea y agrupa los vuelos que se le atribuyen.
Declaracion	declaracion.py	Lo que reportó la aerolínea sobre un vuelo; sabe si se presentó a tiempo.

VIGILADO Por el Ministerio de Educación Nacional
Campusvirtual Universidad de Manizales
ISO 9001:2008 MSC-57117129 www.fbcert.services www.certidata.info
Acreditación Institucional de Alta Calidad Resolución 4752 del 15 de mayo de 2019
UNIVERSIDAD DE MANIZALES

Clase	Archivo	Responsabilidad
InformeConciliacion	informe_conciliacion.py	Crea y contiene los hallazgos, los prioriza, los totaliza y puede cerrarse.
Conciliador	conciliador.py	Compara declaraciones contra vuelos operados y ordena registrar cada diferencia.
CargadorCSV	cargador_csv.py	Lee los archivos de datos y construye los objetos del dominio.

Fuente: elaboración propia. Las excepciones (ErrorConciliacion y sus tres hijas) están en excepciones.py.

Una decisión importante fue separar al Conciliador del InformeConciliacion. El primero decide *qué* diferencia existe, pero no guarda datos ni crea hallazgos; el segundo guarda, prioriza y totaliza, pero no compara. Esa separación evita una clase gigante que hiciera todo y facilita probar cada parte por separado.

3.3. La clase abstracta: por qué Hallazgo no debe instanciarse

Se definió Hallazgo como clase abstracta, haciéndola heredar de ABC y decorando tres métodos con @abstractmethod: calcular_valor_en_riesgo(), impacto() y recomendar_accion(). La razón es conceptual: una "diferencia" sin tipo no existe. Siempre es un vuelo sin declaración, una declaración sin vuelo o una diferencia de pasajeros, y un objeto Hallazgo genérico no tendría forma de calcular su valor ni de saber qué hacer. Al hacerla abstracta, Python impide crear objetos incompletos. En la sección 2 de main.py intentamos instanciarla y capturamos el TypeError; además, una prueba automática comprueba que una clase hija que implemente solo uno de los tres métodos abstractos tampoco puede instanciarse.

La clase abstracta cumple una función real en el sistema y no es decorativa: tanto InformeConciliacion como main.py trabajan contra la interfaz que ella define. Para ordenar los hallazgos por valor o para sumarlos por impacto, el informe solo necesita llamar a calcular_valor_en_riesgo() e impacto(), sin saber de qué clase concreta es cada objeto. La actividad no exige una cantidad determinada de métodos abstractos; elegimos tres porque son exactamente las tres cosas que cambian de un tipo de hallazgo a otro, y dejamos como método concreto lo que es común a todos, describir().

3.4. La herencia: por qué representa una relación "es un"

Las clases VueloSinDeclaracion, DeclaracionSinVuelo y DiferenciaPasajeros heredan de Hallazgo. La relación "es un" se cumple en los tres casos: un vuelo sin declaración *es un* hallazgo, igual que una declaración sin vuelo o una diferencia de pasajeros. También se cumple el principio de sustitución, porque en cualquier lugar donde se espera un Hallazgo —la lista del informe o el ciclo de main.py— puede usarse cualquiera de las tres clases sin cambiar el código. No creamos la herencia solo para cumplir el requisito: las tres hijas comparten una base real y se diferencian en datos y reglas reales.

VIGILADO Por el Ministerio de Educación Nacional
Campusvirtual Universidad de Manizales
ISO 9001:2008 MSC-57117129 www.fbcert.services www.certidata.info
Acreditación Institucional de Alta Calidad Resolución 4792 del 15 de mayo de 2019
UNIVERSIDAD DE MANIZALES

Lo que comparten las tres hijas, y por eso vive en Hallazgo, son los atributos privados __codigo_vuelo, __fecha y __tarifa_por_pasajero (con sus propiedades de solo lectura), el constructor que los valida y el método describir(). Lo propio de cada hija, en cambio, es lo que justifica que existan por separado, como muestra la tabla 2.

Tabla 2. Atributos y comportamientos propios de cada clase hija

Elemento	VueloSinDeclaracion	DeclaracionSinVuelo	DiferenciaPasajeros
Atributos propios	__pasajeros_operados	__pasajeros_declarados	__pasajeros_declarados y __ pasajeros_operados; propiedad calculada diferencia; método privado __subdeclarado()
Valor en riesgo	pasajeros operados × tarifa	pasajeros declarados × tarifa	diferencia (valor absoluto) × tarifa
Impacto	Faltante de recaudo	Posible cobro en exceso	Faltante si operaron más de los declarados; exceso en el caso contrario
Acción recomendada	Solicitar la declaración del vuelo	Verificar si el vuelo ocurrió y, si no, anular la declaración	Pedir corregir la declaración, o verificar el conteo y evaluar una posible devolución, según el sentido
Validación propia	pasajeros operados mayores que cero	pasajeros declarados mayores que cero	declarados y operados no pueden ser iguales: sin diferencia no hay hallazgo

Fuente: elaboración propia a partir de los módulos de las tres clases hijas.

3.5. Los métodos sobrescritos y cómo cambia su comportamiento

Las tres hijas sobrescriben cuatro métodos de Hallazgo: calcular_valor_en_riesgo(), impacto(), recomendar_accion() y describir(). El nombre y la forma de llamarlos son idénticos, pero el resultado cambia porque cada hija usa sus propios datos. Con la misma tarifa de $20.000 por pasajero, el programa produce, por ejemplo: para el vuelo ALF102 (120 pasajeros operados sin declaración), $2.400.000 de faltante; para BRV299 (40 pasajeros declarados de un vuelo que no operó), $800.000 de posible cobro en exceso; para BRV201 (declarados 90, operados 95), $100.000 de faltante; y para BRV202 (declarados 75, operados 70), $100.000 de posible cobro en exceso. Los dos últimos casos son del mismo tipo y valen lo mismo, pero se distinguen por su impacto y por la acción recomendada, que dependen del sentido de la diferencia.

El método describir() merece una mención aparte: en Hallazgo es concreto y entrega vuelo, fecha y tarifa aplicada; cada hija lo sobrescribe llamando primero a super().describir() y agregando después su información propia, por ejemplo "Operado sin declaración: 120 pasajeros". De esa manera reutilizamos la lógica común en lugar de repetirla.

3.6. Cómo se evidencia el polimorfismo en el programa principal

En la sección 3 de main.py, el informe entrega con hallazgos_priorizados() una única colección que mezcla objetos de las tres clases hijas. Un solo ciclo for la recorre y, en cada vuelta, invoca los mismos cuatro métodos sobre el objeto —describir(), calcular_valor_en_riesgo(), impacto() y recomendar_accion()—. El ciclo no contiene

VIGILADO Por el Ministerio de Educación Nacional
Campusvirtual Universidad de Manizales
ISO 9001:2008 MSC-57117129 CERTIFIED www.fbcert.services www.certidata.info
Acreditación Institucional de Alta Calidad Resolución 4792 del 15 de mayo de 2019
UNIVERSIDAD DE MANIZALES

ningún condicional, ni isinstance, ni type: es Python quien decide, en tiempo de ejecución, qué versión de cada método ejecutar según la clase real del objeto. La salida del programa muestra los seis hallazgos en una misma lista, cada uno con su descripción, su valor y su acción propios.

Se escribió una prueba automática que lee el código de main.py con el módulo ast y comprueba que existe un único ciclo que llama a calcular_valor_en_riesgo() y que dentro de él no hay condicionales, comparaciones, bloques try ni comprobaciones de tipo. Otra prueba verifica que informe_conciliacion.py, donde se ordenan y se suman los hallazgos, tampoco usa isinstance ni type.

3.7. Relaciones entre objetos y composición

El diseño incluye cuatro tipos de relación entre objetos, que procuramos no confundir. Hay **asociaciones**: Vuelo guarda el Aeropuerto donde operó (atributo __aeropuerto, validado en el constructor y usado por cobro_esperado()), y Declaracion guarda la Aerolinea que la presentó. También son asociaciones las referencias que Conciliador y CargadorCSV mantienen con el Aeropuerto. En todas, un objeto conoce a otro pero no lo crea ni lo contiene.

Hay una **agregación** entre Aerolinea y Vuelo. La aerolínea guarda una lista privada de vuelos, pero no los crea: los construye CargadorCSV y se le entregan mediante agregar_vuelo(). Es agregación y no composición porque los vuelos tienen vida propia; una prueba lo demuestra eliminando la aerolínea y comprobando que el vuelo sigue existiendo.

La **composición** está entre InformeConciliacion y Hallazgo. El informe no recibe hallazgos ya construidos: sus métodos registrar_vuelo_sin_declaracion(), registrar_declaracion_sin_vuelo() y registrar_diferencia_pasajeros() reciben datos simples y fabrican el hallazgo por dentro. Por eso un hallazgo no tiene sentido fuera del informe que lo originó: nace dentro de él y desaparece con él. Lo comprobamos con una prueba que usa una referencia débil (weakref) y verifica que, al eliminar el informe, sus hallazgos dejan de existir. Esta relación forma parte esencial de la estructura del informe, que es en esencia el conjunto de sus hallazgos.

No resolvimos esta relación con herencia porque la pregunta "es un" tiene respuesta negativa: un informe *no es* un hallazgo, sino que *tiene* hallazgos. Hacer que InformeConciliacion heredara de Hallazgo habría mezclado el comportamiento de una diferencia individual con el de una colección, y habría obligado al informe a implementar métodos de valoración que no le corresponden. Una versión inicial de nuestro diseño hacía que el Conciliador creara los hallazgos y se los pasara al informe; la descartamos porque eso habría sido agregación y no composición.

3.8. Encapsulamiento y constructores

Todos los atributos de las clases son privados (prefijo __) y se leen mediante propiedades de solo lectura. Intentar modificar desde fuera vuelo.pasajeros_operados produce un AttributeError, como se demuestra en la sección 4 de main.py. Las colecciones internas

VIGILADO Por el Ministerio de Educación Nacional
Campusvirtual Universidad de Manizales
ISO 9001:2008 MSC-57117129 www.fbcert.services www.certidata.info
Acreditación Institucional de Alta Calidad Resolución 4792 del 15 de mayo de 2019
UNIVERSIDAD DE MANIZALES

—los vuelos de una aerolínea, los hallazgos de un informe— se devuelven como tuplas, de modo que quien las recibe puede recorrerlas pero no alterarlas; de hecho, intentar agregar un elemento a aerolinea.vuelos también falla con AttributeError.

Los constructores validan los datos antes de guardarlos, apoyándose en las funciones de validaciones.py: un texto no puede estar vacío, un código solo admite letras, números y guiones, los pasajeros deben ser enteros positivos, las fechas deben ser objetos de fecha y una declaración no puede presentarse antes de que ocurra el vuelo. Si algún dato es inválido se lanza DatosInconsistentesError, de manera que ningún objeto del sistema puede quedar creado en un estado inconsistente. En el mismo archivo normalizamos los códigos a mayúsculas, para que "alf101" y "ALF101" se traten como el mismo vuelo.

3.9. Las excepciones personalizadas y la situación que controla cada una

Identificamos tres situaciones anormales propias del dominio y creamos una excepción para cada una. Las tres heredan de una clase común, ErrorConciliacion, lo que permite capturarlas por separado o todas juntas. Cada excepción se lanza desde el método responsable de detectar el problema, se captura en main.py por su nombre exacto —nunca con un except Exception genérico— y, tras cada captura, el programa muestra un mensaje claro y demuestra que el sistema continúa funcionando.

Tabla 3. Excepciones personalizadas

Excepción	Se lanza en	Situación que controla	Caso demostrado en main.py
DeclaracionExtemporaneaError	Declaracion.verificar_plazo()	La declaración se presentó más de 3 días (plazo ficticio) después del vuelo.	La declaración del vuelo CHR304 se presentó 8 días tarde; el programa continúa con las demás.
InformeCerradoError	InformeConciliacion.verificar_abierto(), invocado por cerrar(), los tres registrar_* y Conciliador.conciliar()	Se intenta modificar un informe que ya fue cerrado.	Se cierra el informe y se intenta registrar otro hallazgo; el informe conserva sus 6 hallazgos y sigue consultable.
DatosInconsistentesError	Funciones de validaciones.py en los constructores, y CargadorCSV	Un dato es inválido, vacío, de tipo incorrecto o contradictorio.	Un CSV con -90 pasajeros en la línea 4 se rechaza completo; luego el archivo correcto sí se carga.

Fuente: elaboración propia. Los tres casos se ejecutan en la sección 5 de main.py.

Tomamos dos decisiones que decidimos explicar. La primera es que una declaración tardía *sí puede crearse*: el error no se lanza en el constructor sino al llamar a verificar_plazo(). Si saliera al construirla, una sola declaración tardía impediría cargar los datos y el sistema se detendría, y lo que pide la actividad es justamente que el programa pueda seguir. La segunda es que los errores de lectura de archivos (archivo inexistente,

VIGILADO Por el Ministerio de Educación Nacional
Campusvirtual Universidad de Manizales
ISO 9001:2008 MSC-57117129 CERTIFIED www.fbcert.services www.certidata.info
Acreditación Institucional de Alta Calidad Resolución 4792 del 15 de mayo de 2019
UNIVERSIDAD DE MANIZALES

codificación incorrecta, formato inválido) los convertimos en DatosInconsistentesError, en lugar de crear una cuarta excepción. Reconocemos que es una ampliación del significado de esa excepción, pero mantiene el diseño simple y los mensajes indican siempre el archivo y la línea del problema.

3.10. Los datos y la carga desde archivos

Decidimos que el sistema leyera sus datos desde dos archivos CSV ficticios, vuelos_operados.csv y declaraciones.csv, para que se pareciera más a un sistema real sin añadir dependencias externas: solo usamos la biblioteca estándar de Python. CargadorCSV aplica una carga estricta y atómica: si una sola fila es inválida, rechaza el archivo completo y no deja nada a medias, indicando archivo y línea. Elegimos ese comportamiento porque es más fácil de explicar y de verificar que una carga tolerante que "salte" filas dañadas y entregue resultados parciales sin que nadie lo note. Un archivo con solo el encabezado también se rechaza, porque conciliar contra un archivo vacío marcaría todos los vuelos como "sin declaración" y daría un resultado engañoso.

3.11. Decisiones que modificamos durante el desarrollo

El diseño final no es el primero que consideramos, y varios cambios surgieron de detectar errores en nuestras propias propuestas. La primera idea para el problema era una simple calculadora de cobros por vuelo; se descartó porque no resolvía un problema de fondo y quedaba como "un aplicativo más", y reemplazamos por la conciliación entre lo declarado y lo operado. por eso cada una se prueba con un caso propio en main.py.

Inicialmente Hallazgo tenía solo dos comportamientos que variaban, pero al escribir las hijas notamos que VueloSinDeclaracion y DeclaracionSinVuelo calculaban el valor con la misma fórmula, lo que habría hecho débil la sobrescritura. Por eso se agregó el método impacto(), que se localizó por su significado y que más adelante permite al informe sumar por separado el faltante y el cobro en exceso sin condicionales por tipo. Otros ajustes fueron que Vuelo no guarda a su aerolínea (para evitar una referencia circular con Aerolinea), que un informe que ya tiene hallazgos se rechaza en una nueva conciliación (para no duplicar resultados) y que un mismo código de vuelo y fecha en dos aerolíneas distintas se rechaza por ambiguo.

4. Declaración de Uso de Herramientas de Inteligencia Artificial

Se utilizó una herramienta de inteligencia artificial como apoyo para verificar el correcto funcionamiento del proyecto, aplicar buenas prácticas en el desarrollo de software y comprender diferentes conceptos de POO.

5. Conclusiones

Construir este sistema demostró que el valor de la programación orientada a objetos no está en usar sus mecanismos, sino en usarlos donde corresponden. La clase abstracta Hallazgo tiene sentido porque una diferencia sin tipo no existe; la herencia es legítima porque las tres hijas se sustituyen entre sí; y la composición entre InformeConciliacion y

VIGILADO Por el Ministerio de Educación Nacional
Campusvirtual Universidad de Manizales
ISO 9001:2008 MSC-57117129 CERTIFIED www.fbcert.services www.certidata.info
Acreditación Institucional de Alta Calidad Resolución 4792 del 15 de mayo de 2019
UNIVERSIDAD DE MANIZALES

Hallazgo se justifica porque el hallazgo nace y muere con su informe. Cuando intentamos aplicar estos conceptos por obligación, como al hacer que el conciliador creara los hallazgos, el diseño se volvía incorrecto y tuvimos que rectificarlo.

Las principales dificultades fueron tres. Distinguir con rigor la agregación de la composición exigió pensar en quién crea los objetos y en qué ocurre con ellos cuando desaparece el contenedor. Lograr un polimorfismo sin condicionales nos obligó a mover las diferencias de comportamiento hacia las clases hijas, lo que terminó dando un diseño más limpio, con métodos como impacto(). Y hacer que el sistema siguiera funcionando ante cada excepción nos llevó a decidir con cuidado en qué método se lanza cada una y dónde se captura.

Entre los aprendizajes, el más importante es que verificar es parte del diseño: las pruebas automáticas y, sobre todo, las alteraciones deliberadas del código nos permitieron descubrir errores que ninguna lectura habría detectado. También aprendimos que el uso de una herramienta de inteligencia artificial acelera el trabajo, pero que sus propuestas deben cuestionarse; varias de las mejores decisiones de este proyecto nacieron de corregir una propuesta inicial. Como limitaciones, el modelo trabaja con un solo aeropuerto y con reglas ficticias, y asume que las declaraciones entregadas al conciliador corresponden a ese aeropuerto. Esas simplificaciones son deliberadas y pueden ampliarse en trabajos futuros.

6. Referencias Bibliográficas

Python Software Foundation. (s. f.). *abc — Abstract Base Classes*. En *The Python Standard Library*. https://docs.python.org/3/library/abc.html

Python Software Foundation. (s. f.). *ast — Abstract Syntax Trees*. En *The Python Standard Library*. https://docs.python.org/3/library/ast.html

Python Software Foundation. (s. f.). *csv — CSV File Reading and Writing*. En *The Python Standard Library*. https://docs.python.org/3/library/csv.html

Python Software Foundation. (s. f.). *unittest — Unit testing framework*. En *The Python Standard Library*. https://docs.python.org/3/library/unittest.html

Universidad de Manizales. (2026). *Actividad colaborativa N°3. Proyecto final* [Documento de la actividad]. Facultad de Ciencias e Ingenierías.

VIGILADO Por el Ministerio de Educación Nacional

Campusvirtual Universidad de Manizales

CERTIFIED ISO 9001:2008 MSC-57117129 www.fbcert.services www.certidata.info

Acreditación Institucional de Alta Calidad Resolución 4792 del 15 de mayo de 2019

UNIVERSIDAD DE MANIZALES

Anexo. Figura 1. Diagrama UML de clases del sistema

